

Reviewer Pack — Minimal Index of Tropical Grothendieck Groups and SAT Clause...

Entry 956cb0f66537 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 08:00:26 UTC

Minimal Index of Tropical Grothendieck Groups and SAT Clause Set Complexity

Entry ID: 956cb0f66537 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-03 08:00:26 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry (Tropical Grothendieck Theory) **Field B** (complexity object): Boolean Satisfiability (SAT) Clause Sets

Statement:

For every CNF φ with n variables, the minimal index of the tropical Grothendieck group associated with φ is linearly correlated with the number of clauses in φ , such that $\text{Index_G}(\varphi) = \Theta(|C(\varphi)|)$, where $|C(\varphi)|$ denotes the clause count and $\text{Index_G}(\varphi)$ is computed using a constructive mapping from φ to the tropical Grothendieck group via tropicalization of its characteristic polynomial modulo 2.

Rationale (proposer's reasoning):

The use of tropical Grothendieck groups provides a novel way to encode combinatorial information, potentially revealing hidden structures in SAT clause sets. If a correlation exists between the index of these groups and clause count, it may expose a new complexity-theoretic insight into the structure of SAT instances.

Taxonomy category: TROPICAL_GROTHENDIECK (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 381217ccd068c93e

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient between the minimal index of the tropical Grothendieck group (Index_G) and the number of clauses ($|C(\varphi)|$) for all generated CNFs is ≥ 0.8 , with no seed producing a correlation coefficient < 0.5 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.2s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n: int, num_clauses: int):
        cnf = []
        for _ in range(num_clauses):
            clause = [random.randint(1, n), random.randint(-n, -1)]
            cnf.append(clause)
        return cnf

    def characteristic_polynomial(cnf):
        n = len(cnf[0])
        matrix = [[Fraction(0) for _ in range(n)] for _ in range(n)]
        for clause in cnf:
            literal = abs(clause[0]) - 1
            if clause[0] > 0:
                matrix[literal][literal] += Fraction(1)
            else:
                matrix[literal][literal] -= Fraction(1)
        return matrix

    def tropicalize(matrix):
        n = len(matrix)
        for row in range(n):
            for col in range(n):
                if matrix[row][col] < 0:
                    matrix[row][col] = -matrix[row][col]
        return matrix

    def compute_index(tropical_matrix):
        n = len(tropical_matrix)
        for i in range(n):
            tropical_matrix[i][i] += Fraction(1)
```

```

        return sum(sum(row) for row in tropical_matrix)

def solve(lits_true, lits_false, cls):
    if not lits_true and not lits_false:
        return True
    lit = lits_true[0]
    new_lits_true = [l for l in lits_true if l != lit] + [other_lit for other_lit in lits_false if other_lit != lit]
    new_lits_false = [l for l in lits_false if l != lit] + [other_lit for other_lit in lits_true if other_lit != lit]
    return solve(new_lits_true, cls) or solve(new_lits_false, cls)

def is_satisfiable(cnf):
    literals = set(abs(lit) for clause in cnf for lit in clause)
    return solve(list(literals), [], cnf)

results = []
n_values = [5, 10, 15, 20, 30, 40]
for n in n_values:
    for _ in range(5):
        num_clauses = random.randint(n // 2, n)
        cnf = generate_cnf(n, num_clauses)
        results.append(cnf)

indices = [compute_index(tropicalize(characteristic_polynomial(cnf))) for cnf in results]
clause_counts = [len(cnf) for cnf in results]

if not indices or not clause_counts:
    return {
        "metric_name": "Index_G",
        "metric_value": 0,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

correlation = sum((indices[i] - mean_indices) * (clause_counts[i] - mean_clauses) for i in range(len(indices)))
mean_indices = sum(indices) / len(indices)
mean_clauses = sum(clause_counts) / len(clause_counts)

return {
    "metric_name": "Index_G",
    "metric_value": correlation,
    "instances_tested": len(results),
    "n_max": max(n_values),
    "conjecture_holds": 0.5 <= correlation < 0.8,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")

```

```

results.append(trial_result)

if all(result["conjecture_holds"] for result in results):
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)
    print(f"RESULT: SUPPORTED mean={sum(result['metric_value'] for result in results) / len(results)}")
else:
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_25bd7618.py", line 105, in <module>
    trial_result = run_trial(seed)
                   ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_25bd7618.py", line 73, in run_trial
    indices = [compute_index(tropicalize(characteristic_polynomial(cnf))) for cnf in results]
                ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_25bd7618.py", line 34, in characteristic_polynomial
    matrix[literal][literal] += Fraction(1)
    ~~~~~^~~~~~
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing any data, which prevents us from evaluating the Pearson correlation coefficient required to support or falsify the conjecture. | next: Re-run the test without errors to collect the necessary data for calculating the correlation coefficients.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	18202
2	propose	ollama_remote	glm4:latest	0	0	14137
3	preregistration	ollama_remote	glm4:latest	0	0	20164
4	novelty	ollama_remote	glm4:latest	0	0	8684

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
5	novelty	ollama_remote	glm4:latest	0	0	8604
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14923
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15597
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	24296
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22433
10	judge	ollama_remote	glm4:latest	0	0	17596

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 164636 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/956cb0f66537.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/956cb0f66537.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/956cb0f66537.tar.gz` (if generated)